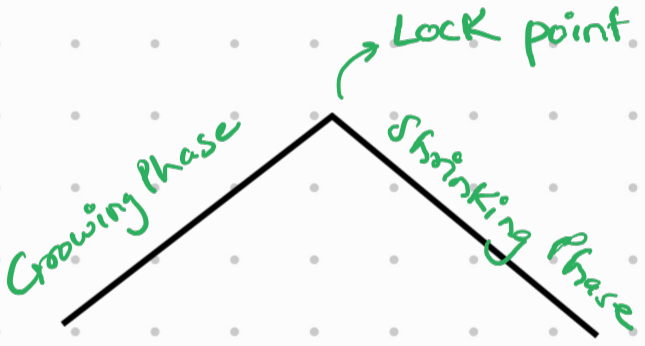
Basic 2PL (2 phase Locking)

Phase 1 : Growing Phase

- Transaction requests for lock from lock manager
- Lock manager either grants or denies the lock request (if other transaction has already placed lock)

Phase 2 : Shrinking Phase

- Transaction cannot acquire any new locks.
- Transaction is only allowed to release locks.



time	T_1	T_2
t_0	Begin	
t_1	X(A)	
t_2	X(B)	
t_3	do some work	
t_4	Unlock (A)	Begin
t_5	Unlock (B)	X(B)
t_6	Commit	X(A)
		Commit

3 big issues with Basic 2PL

1) Deadlock

T_1	T_2
Begin	
lock - X(A)	
	Begin
	lock - X(B)
	lock - X(A)
	(wait for T_1 to release A)
lock - X(B)	
(wait for T_2 to release B)	

- T_1 is waiting for T_2 to release B.
- T_2 is waiting for T_1 to release A.

Strategies to get out of deadlock :-

1) Timeout

- When the transaction scheduler sees that the transaction is waiting too long for the lock, it simply assumes a deadlock and aborts it.

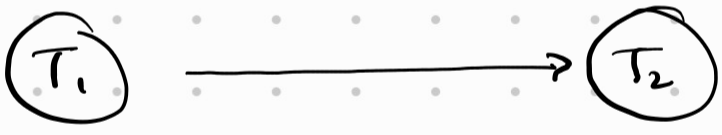
- The problem with this is what if there was no deadlock and the T_2 was simply taking a lot of time to finish.

T_1 ----> T_2 (Taking too long)

- Another problem is how do we decide the timeout period.

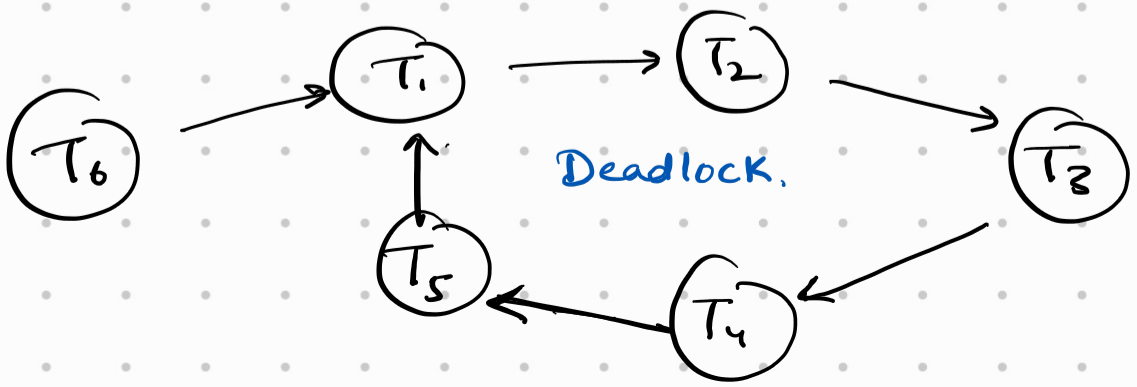
2) Wait-for-Graph (WFG)

- Transaction scheduler creates this graph.
- Whenever, a lock is released, Scheduler will delete the edge.



T_1 is waiting for T_2 to release some lock.

- Periodically, the scheduler will check for cycle in the graph.



- When a cycle exists in WFG, that means deadlock exists.
- In order to resolve this deadlock, we choose one (or more) transactions and abort them.

- How does the scheduler decides which transaction should be aborted?

  The scheduler looks decides based on the following :-

  1) The amount of effort txn has put till now.

  2) The amount of effort required to finish txn.

  3) Cost of aborting the txn (how many updates has been already done).

  4) The number of cycles that contains the transaction.

3) Use Conservative 2PL

4) Timestamp based Deadlock prevention

- The idea is to assign each txn a timestamp when the txn enters into the system/ begins.
  So T_1 which enters first will get a smaller timestamp than T_2 which enters later into the system.

- The system uses timestamps only to decide whether a transaction should wait or rollback.

- If a transaction is rolled back, it retains its old timestamp when restarted – this ensures every transaction gets its turn and doesn't starve indefinitely.

Two different schemes are there :-

1) Wait-die scheme : It is a non-preemptive approach.

- When transaction T_i requests a data item currently held by T_2, T_i is allowed to wait only if it has a timestamp smaller than that of T_2, otherwise it is rolledback.
- So basically, Old txn waits for the new txn, but new txn cannot wait for old txn and is rolledback.
- The older a transaction gets, the more it tends to wait.

2) Wound-wait scheme : It is a preemptive approach. When transaction T_i requests a data item currently held by T_2, T_i will abort T_2 if T_i is older, and get the data item held by T_2. If T_i is younger/newer, it will wait for T_2.

So, basically, old txn wounds new txn and aborts it. While new transaction waits for the old one.

- Older txn will not wait for new ones, hence there may be fewer rollbacks here.

- In both the scheme, older transaction is never rolled back. If either waits (wait-die) or wounds (wound-wait).
- Since a rolledback transaction is always assigned the same timestamp, with passage of time, a time comes when it has the smallest timestamp in the system, hence making sure it never starves.

* Problems with both the schemes is that unnecessary rollbacks occur, even if there may be no deadlock, hence, losing progress.

2) Less degree of concurrency.

Even if a transaction has already performed all the operations on a data item Q, txn cannot release it because txn has not entered its shrinking phase yet. So, some other txn will have to wait.

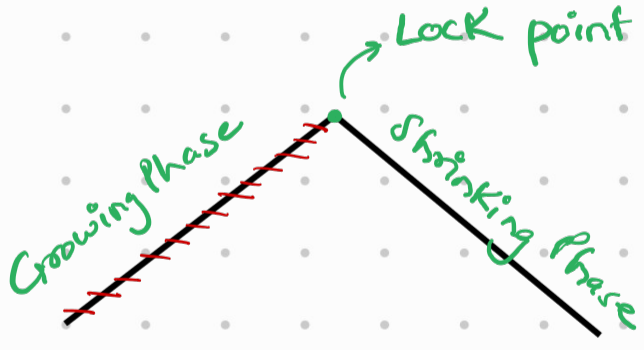
3) Dirty Reads & Cascading Rollbacks

T_1	T_2
lock - x(A)	
R(A)	
W(A)	
Unlock(A)	
⋮	lock - S(A)
	R(A) → Dirty Read
	⋮

- If T_2 commits before T_1, and T_1 fails, then T_1 will be able to rollback, but T_2 will not be able to do so as it has already committed.
- Now let's suppose T_2 has not committed yet, and T_1 fails. In this case T_1 will rollback. Also, since T_2 performed dirty read from T_1, we will need to rollback T_2 also. This is called cascading rollback.
- Cascading rollbacks are solved in strict 2PL & rigorous 2PL.

Conservative 2PL / Static 2PL

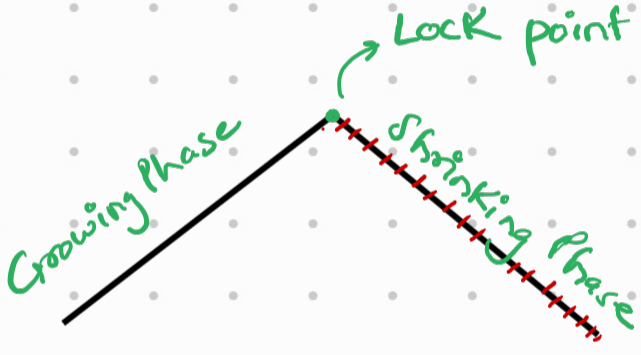
- This version of 2PL tries to eliminate deadlock that occurred in Basic 2PL, but may starve.
- There is no growing phase here.
- Each txn will try to acquire all the locks at the start of the txn itself. If all locks are not available, it will be not be granted any lock and must wait.
- The txn must know beforehand what data items will be required during execution.



- less concurrent.
- Deadlock is solved cuz a transaction will not proceed unless it acquires all the locks. Either it acquires all the locks it requires or none at all. There is no partial acquisition of locks which generally is the root cause of deadlocks.
- Can still suffer from dirty reads & cascading aborts.

Rigorous 2PL

- This a version of 2PL that eliminates cascading rollbacks & dirty reads.
- The idea is a transaction acquires locks, but does not perform unlocking until transaction commits.
- So, there is no shrinking phase, when the txn commits, then all the locks held are automatically released.



- If we lock a data item Q till commit, that means some other transaction will not be able to acquire a lock on Q and so, it won't be able to read Q, and hence no dirty read and no cascading aborts.
- This still suffers from deadlock. Deadlock is acceptable, as we can recover from it and the chances of deadlock is very less.
- Degree of concurrency is less as transaction can't release a data item even if the work is done, and some other txn will have to wait longer.

Strict 2PL

- This is an improvement over Rigorous 2PL.
- In rigorous 2PL, both shared and exclusive locks are held till commit.
  In Strict 2PL, we are allowed to release shared lock in the shrinking phase. Exclusive locks are still held till commit.
- The goal was to eliminate dirty reads. If a data item Q has a shared lock on it, that means the current transaction can't modify it. So even if some other transaction gets a shared lock on it, it doesn't matter cuz Q was not modified, and hence it cannot be dirty read.
- As for X-locks, they must be held till the end, because the current txn might have updated the value, and if we release it before commit then some other txn may read this updated value leading to dirty reads.
- The problem of deadlock is still there. We can use different deadlock prevention/recovery strategies for this.
- This is better than rigorous 2PL in terms of degree of concurrency as we are now allowing shared locks to be released before commit.